

Lab Assignment 2 Report

a)

The purpose of this assignment was to create a program that will read in integers from a file up to 3 digits, sum the integers, and then print out the contents. The way I went about this starting out by reading the contents of the file into a buffer. The buffer should then hold all of the integers, separated by a new line character. Then I wrote code that would iterate through the contents in the buffer byte by byte. It would read in a byte and add it to a temp buffer. Then, it would keep doing that until a newline was reached. The temp buffer should then hold an integer. Then the code to process those integers would do a series of multiplication by 10 and addition to get the correct integer value. Then that integer would be added to an array. This would keep repeating until the buffer was empty. Then I looped through the array, summing all of the integer. Lastly, the final sum was converted to a string and printed out. I do have an issue where a few lines from the text file will print out after the sum that I can't seem to solve. Ultimately, the code works fairly well with a few issues here and there.

b)

section .data

newline db 0x0A ; Newline character for output formatting

pathname db "randomInts.txt", 0 ; File path (null-terminated string)

result_msg db 'Sum of numbers: ', 0 ; Message to print before the sum

result_buffer db 20, 0 ; Buffer to store the ASCII string result (max 10 digits)

section .bss

buffer resb 1024 ; Buffer to store the file contents

array resd 1000 ; Reserve space for storing integers (max 1000 integers)

temp resb 10 ; Temp buffer to store up to 10 digits (for a number)

temp_len resb 1 ; To keep track of current length of digits in temp

section .text

global _start

_start:

; Open the file

mov eax, 5 ; sys_open

lea ebx, [pathname] ; File name address

mov ecx, 0

int 0x80

mov ebx, eax

test ebx, ebx

jz done

jmp read

read:

; Read the file into buffer

mov eax, 3 ; sys_read

lea ecx, [buffer]

mov edx, 1024

int 0x80

test eax, eax

jz done

lea esi, [buffer]

jmp parse

parse:

```
lea esi, [buffer]
```

```
lea edi, [array]
```

```
xor ecx, ecx
```

```
xor edx, edx
```

```
jmp iterate
```

iterate:

```
mov al, [esi]
```

```
cmp al, 0
```

```
je done
```

```
cmp al, [newline]
```

```
je process
```

```
; If it's a digit, convert it to an integer and store it in the temporary buffer
```

```
cmp al, '0'
```

```
jle not_digit
```

```
cmp al, '9'
```

```
jg not_digit
```

```
; It's a digit, store it in the temp buffer
```

```
mov [temp + edx], al
```

```
inc edx      ; Increment temp buffer length
```

```
; If we have collected digits, process the number
```

```
cmp edx, 10
```

```
jl not_digit
```

process:

```
; Convert the temp buffer (up to 10 digits) to an integer
```

```
xor eax, eax
```

```
xor ebx, ebx
```

```
mov ecx, edx
```

```
; Start converting digits from the left (most significant digit first)
```

```
xor edx, edx
```

convert_digits:

```
movzx edx, byte [temp + ebx] ; Load the current digit (ASCII)
```

```
sub dl, '0' ; Convert from ASCII to integer
```

```
; Multiply the accumulated result by 10 (shifting left by 1 decimal place)
```

```
imul eax, eax, 10
```

```
add eax, edx
```

```
inc ebx
```

```
cmp ebx, ecx
```

```
jl convert_digits
```

```
; Store the result in the array
```

```
mov [edi], eax
```

```
add edi, 4
```

; Reset temp buffer

xor edx, edx

mov byte [temp], 0

inc esi

jmp iterate

not_digit:

inc esi

jmp iterate

done:

; Sum the integers in the array

xor eax, eax

lea esi, [array]

mov ecx, 1000

sum_loop:

cmp ecx, 0 ; Check if we have processed all integers

je print_result ; If all integers are summed, print the result

add eax, [esi]

add esi, 4

dec ecx

jmp sum_loop

print_result:

; Convert the sum (in eax) to a string

mov ebx, eax ; Copy the sum to ebx

lea edi, [result_buffer] ; Load address of result_buffer to store the string

add edi, 9 ; Point to the last character of the result buffer

mov byte [edi], 0 ; Null-terminate the string

dec edi ; Move back to the last position for digits

; If the sum is zero, just write '0'

test ebx, ebx

jz write_zero

convert_sum_to_string:

xor edx, edx ; Clear edx to prepare for division

mov ecx, 10 ; Divisor for converting to base 10

div ecx ; Divide eax by 10, quotient in eax, remainder in edx

add dl, '0' ; Convert remainder to ASCII character

mov [edi], dl ; Store the character in result_buffer

dec edi ; Move to the next character

test eax, eax ; Check if the quotient is 0

jnz convert_sum_to_string ; If not, continue converting

; Print result message

mov eax, 4

```
mov ebx, 1
lea ecx, [result_msg]
mov edx, 16
int 0x80
```

```
; Print the sum (converted to a string)
```

```
mov eax, 4
mov ebx, 1
lea ecx, [edi + 1]
mov edx, 10
int 0x80
```

```
; Print a newline
```

```
mov eax, 4
mov ebx, 1
lea ecx, [newline]
mov edx, 1
int 0x80
```

```
; Exit the program (for Linux system call)
```

```
mov eax, 1      ; syscall number for exit
xor ebx, ebx    ; exit code 0
int 0x80        ; invoke syscall
```

```
write_zero:
```

```
mov byte [edi], '0'
```

```
lea ecx, [edi]
```

```
mov eax, 4
```

```
mov ebx, 1
```

```
mov edx, 1
```

```
int 0x80
```

```
jmp done
```

c)

randomInt100 output:

```
22     mov ebx, eax
23     test ebx, ebx
24     jz done
25
26     jmp read

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

[kalvey1@linux4 lab_assignment_2]$ nasm -f elf64 read.asm -o read.o
[kalvey1@linux4 lab_assignment_2]$ ld read.o -o read
[kalvey1@linux4 lab_assignment_2]$ ./read
Sum of numbers: 4679
13
[kalvey1@linux4 lab_assignment_2]$
```

other file file:

```
SH FS - umbcgl > cmpe310 > lab_assignment_2 > randomInts.txt
1 100
2 1
3 2
4 3
5 4
6 5
7 6
8 7
9 8
10 9
11 9
12 8
13 7
14 6
15 5
16 4
17 3
18 2
19 1
20 100
21
```


output:

```
24      jz done
25
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL + v
[kalvey1@linux4 lab_assignment_2]$ nasm -f elf64 read.asm -o read.o
[kalvey1@linux4 lab_assignment_2]$ ld read.o -o read
[kalvey1@linux4 lab_assignment_2]$ ./read
Sum of numbers: 290
2
3
[kalvey1@linux4 lab_assignment_2]$
```